

BUS ROUTE AND TICKETING MANAGEMENT SYSTEM

TEST PLAN

1. Purpose

This test plan explains how we will check that our system works correctly, handles incorrect input and keeps related information consistent.

The system includes route, trip, passenger, bus pass and ticket management. Testing will cover both individual functions and the way these parts work together.

2. Testing Objectives

Our testing will check whether the system can:

- Perform the main operations correctly.
- Reject invalid or incomplete input.
- Keep IDs unique and records consistent.
- Protect relationships between routes, trips, passengers, passes and tickets.
- Handle changes such as updates, cancellations and renewals correctly.
- Save and load information without losing important data.

3. Testing Coverage

Component	Main checks
Routes	Add, search, update and delete routes
Trips	Create, search, update, cancel and check seats
Passengers	Register, search and update passenger details
Bus passes	Issue, validate, renew and change status
Tickets	Purchase, search, price, cancel and reuse seats
Validation	Invalid values, missing records and boundary cases
File handling	Save, load and restore data
Integration	Correct behaviour between connected components

4. Testing Approach

We will use unit, validation, integration, persistence and manual testing.

Unit testing will check individual classes and manager methods. Validation testing will use incorrect values to make sure the system does not accept unreliable data. Integration testing will check how the managers depend on each other. Persistence testing will check whether saved records can be loaded correctly. Manual testing will be used to check the complete application through the main menu.

5. Main Test Conditions

The following conditions will be considered during testing:

Situation	Expected behaviour
A valid record is entered	The record is created or updated
A required value is missing	The operation is rejected
An ID does not exist	A suitable result or error is returned
An invalid date, status or number is entered	The value is rejected
A route is connected to a trip	The relationship is not broken accidentally
A passenger buys a ticket	The passenger and trip must exist
A seat is already occupied	Another ticket cannot use the same active seat
A ticket is cancelled	Its status changes and its seat becomes available
A trip has active tickets	The trip cannot be cancelled
A bus pass is expired or inactive	It cannot be treated as a valid pass
Data is saved and loaded	The important details remain unchanged

6. Important System Relationships

Ticket purchase will be tested as a complete process rather than only as a single method. The system must confirm that the passenger, trip and route exist, that the seat is valid and available, and that any bus pass belongs to the correct passenger.

We will also check that actions in one part of the system affect related records correctly. For example, cancelling a ticket should free its seat, while cancelling a trip with active tickets should be prevented.

7. Validation and Boundary Cases

Testing will include normal values as well as values close to or outside the allowed limits. These include:

- Empty names and destinations
- Invalid phone numbers

- Negative fares or distances
- Zero, negative or invalid IDs
- Missing records
- Invalid date order
- Expired, suspended or cancelled passes
- Full trips
- Occupied seats
- Repeated cancellation or renewal attempts
- Loading data from a missing file

These cases are included because a system may appear to work with correct input but still fail when users make mistakes.

8. Test Environment

The system will be tested using Python and Visual Studio Code. JSON files will be used for data storage. Python test scripts will be used for individual functions, and the main menu will be tested manually.

9. Pass and Fail Criteria

A test will pass when the actual result matches the expected result and the system keeps the data in a correct state.

A test will fail when the system accepts invalid data, changes an unrelated record, gives an incorrect result or produces an unexpected error.

The actual outcomes will be recorded in the separate Test Results document.

10. Test Evidence

Evidence will include test output, screenshots where useful, results from the main application and saved JSON data. The evidence will focus on important operations such as ticket purchase, seat checking, cancellation, validation and data saving.

11. Summary

This test plan considers not only whether each function works, but also what happens when functions are combined or when a record is changed.

The main focus is to protect the correctness of the data and the relationships between the different parts of the system.